

```
// WebServer.h
```

```
#define NODEMCU_FILE
```

```
#include <ESP8266WebServer.h>
```

```
void handleRoot(ESP8266WebServer *server);  
void handleGetInventory(ESP8266WebServer *server);  
void handleAdd(ESP8266WebServer *server);  
void handleDelete(ESP8266WebServer *server);  
void handleClear(ESP8266WebServer *server);  
void WebSetup(ESP8266WebServer *server);
```

```
// WebServer.cpp
```

```
#include "WebServer.h"
```

```
#include "CsvOperations.h"
```

```
#include "NtfyNotifier.h"
```

```
#include ".common/comm.h"
```

```
#include <ESP8266WebServer.h>
```

```
#include <ESP8266mDNS.h>
```

```
#include <SD.h>
```

```
void WebSetup(ESP8266WebServer *server)
```

```
{
```

```
    if (MDNS.begin("tim")) // Why does this stupid thign add  
                           // 2KB of overhead....whyyyy
```

```
        d_SerialPrintln("mDNS responder start!");
```

```
    else
```

```
        d_SerialPrintln("mDNS startup failed!");
```

```
    server->on("/", HTTP_GET, [server]() { handleRoot(server); });
```

```
    server->on("/inventory", HTTP_GET, [server]() { handleGetInventory(server); });
```

```
    server->on("/add", HTTP_POST, [server]() { handleAdd(server); });
```

```
    server->on("/delete", HTTP_POST, [server]() { handleDelete(server); });
```

```
    server->on("/clear", HTTP_POST, [server]() { handleClear(server); });
```

```
    server->begin();
```

```
    d_SerialPrintln("Webserver started!");
```

```
}
```

```
void handleClear(ESP8266WebServer *server)
```

```
{
```

```
    if (SD.exists(INVENTORY_FILE))
```

```
        SD.remove(INVENTORY_FILE);
```

```
    File file = SD.open(INVENTORY_FILE, FILE_WRITE);
```

```
    if (!file)
```

```
    {
```

```
        server->send(500, "text/plain", "Failed to recreate file");
```

```
        return;
```

```

    }

    file.close();
    server->send(200, "text/plain", "Cleared");
}

void handleRoot(ESP8266WebServer *server)
{
    File f = SD.open("/index.html", FILE_READ);
    if (!f)
    {
        server->send(404, "text/plain", "index not found!");
        return;
    }
    server->streamFile(f, "text/html");

    f.close();
}

void handleGetInventory(ESP8266WebServer *server)
{
    File f = SD.open(INVENTORY_FILE, FILE_READ);
    if (!f)
    {
        server->send(500, "text/plain", "File not found");
        return;
    }
    server->streamFile(f, "text/csv");
    f.close();
}

void handleAdd(ESP8266WebServer *server)
{
    if (!server->hasArg("box_id") or !server->hasArg("item") or !server->hasArg("expiry")
or
        !server->hasArg("physical") or !server->hasArg("profile"))
    {
        server->send(400, "text/plain", "Missing fields");
        return;
    }

    InventoryRecordLine record;

    strncpy(record.box_id, server->arg("box_id").c_str(), sizeof(record.box_id));
    strncpy(record.item_name, server->arg("item").c_str(), sizeof(record.item_name));
    record.reg_date = time(nullptr);
    record.expiry_date = server->arg("expiry").toInt();
    record.physical = server->arg("physical") == "1";
    record.condition = FoodCondition::FRESH;
    record.notified = 0;
    record.preset = (ProfileID)(server->arg("profile").toInt());

    AddRecord(&record);

```

```
    NotifyItemAdded(&record);

    server->send(200, "text/plain", "Ok.");
}

void handleDelete(ESP8266WebServer *server)
{
    if (!server->hasArg("box_id"))
    {
        server->send(400, "text/plain", "Missing box_id");
        return;
    }
    DeleteRecord(server->arg("box_id").c_str());
    NotifyItemDeleted(server->arg("box_id").c_str());
    server->send(200, "text/plain", "Ok.");
}
```